

Assignment 5

Convolutional Neural Networks (15P)

Goals

You roughly know how neural networks and specifically CNNs work. You know how to train a neural network and how to use it for predictions. You know how to change hyperparameters for training neural networks. You know how to measure the quality of a neural network's predictions. You can build a simple application that uses a neural network.

1. Exploring Hyperparameters (5P)

Experiment with hyperparameters to explore their influence on the prediction accuracy of a CNN. As a starting point, use the CNN from the notebook we used in the course and the subset of the HaGRID dataset (see ILIAS).

Now, explore the influence of hyperparameters during training on **prediction accuracy** for a test dataset, as well as **inference time for predictions**. The hyperparameters are:

- Image resolution
- Image color mode (only two values are necessary here)
- Batch size
- Number of neurons in the fully connected layers
- Number of training images
- Number of categories
- Activation function of the convolution kernels
- Activation function of the fully connected layers
- Kernel size for the convolution layers
- Number of convolution layers
- Data augmentation (present vs. not present)
- Optimizer
- Dropout Rate
- Train/Test split
- Number of fully connected layers
- Early stopping patience
- Random Contrast intensity
- Random Brightness intensity
- Scale factor for number of conv filters

You will be assigned one of those hyperparameters. Select at least four different values for your hyperparameter, train a network with each, and log performance metrics. Before training, briefly describe your approach and your assumptions in the notebook. After training, report your findings and visualize your results appropriately. We will discuss your findings in the next session.

Score

- (1P) selected hyperparameter values make sense
- (1P) networks were trained correctly
- (1P) there are results
- (2P) results are reported and visualized appropriately

2. Gathering a Dataset (4P)

Extend the HaGRID dataset by capturing one image of hand gestures for each of the following three categories: *like*, *rock*, *peace*. Your images should be similar to those in the dataset.

Then, annotate your dataset and store annotations in a JSON file called *annot-[your_name].json* with a format compatible to the annotations of the original dataset. If there are multiple hands in an image, annotate hands without a gesture as *no-gesture*. You don't have to annotate hand landmarks and so on – only label and bounding box are necessary.

Your tutors also have done this task in the past. Use a CNN trained on (the subset of) the HaGRID dataset to make predictions for your and the tutors images. Plot the results as one confusion matrix and save it as *conf-matrix.png*.

Score

- (1P) sufficient images captured
- (1P) sufficient images annotated
- (1P) annotations are compatible to the HaGRID dataset
- (1P) confusion matrix

3. Gesture-based Media Controls (6P)

Train a classifier that can distinguish at least three different hand poses. Based on this classifier, create a media controller in Python. It should support at least three of the following features: start track, pause track, increase volume, decrease volume, skip track. Media controls can be triggered using the pynput library. In case finding the user's hand in the camera image is a problem, you can for example use a white wall as background or use the cardboard with the four markers to act like a physical bounding box.

Score

- (2P) three hand poses are tracked and distinguished reliably
- (1P) There is a way of handling no gesture, that works
- (1P) three media control features are implemented
- (1P) mapping of gestures to media controls works and makes sense
- (1P) low latency between gesture and the systems reaction

Code Quality: bad code quality and missing documentation can lead to loss of points.